pfi

Python for Fixed Income

COURSE 6 · NEURAL NETWORKS

Case Study: A First Network on Credit Card Default

WEEK 1 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Two halves: the parts, then the case study

Hands-on, sections 5.1 to 5.3

dense_classifier: a network built, and seeded, in a function

cv_network: five folds, each with its own stopping rows and scaler

A threshold from out-of-fold outputs

Why scores come from scikit-learn

Case study, sections 5.4 to 5.11

Two ways to scale, by five folds

How much the seed alone moves a network

A ladder of one setting at a time

Validation rows, with intervals

What it shows, and where it fails

Does a small network rank the accounts better than the models on the desk? Promise nothing; the numbers decide.

How dense_classifier works

1. One function

layers, loss, optimizer in one place

2. Seeded inside

so no network is built unseeded

3. Settings

units, activation, dropout, learning rate

4. Compiled

a Sequential ready to fit

You control

The widths, the activation, the dropout rate, the learning rate.

What comes out

A compiled model with the same starting weights on every build.

A network built in a function

```
def dense_classifier(n_inputs, units=(64, 32), activation="relu", dropout=0.0, learning_rate=1e-3, seed=42):
    keras.utils.set_random_seed(seed)        # first: no network is built unseeded
    ...  Dense(64) -> Dense(32) -> Dense(1, sigmoid); Adam; binary_crossentropy
```

3,393

weights and intercepts for 19 inputs

True

two builds start from the same weights

`Layers: hidden_1, hidden_2, output`

Two runs that differ in one setting must not also differ in where they started.

How cross-validation works

1. Five folds

of the training rows

2. Hold one out

fit on the other four

3. Score it

by a network that never saw it

4. Pool

one honest output per training row

You control

The build, the scaler, the folds, the batch size, the patience.

What comes out

A fold table and the out-of-fold outputs.

cv_network: five folds, each with its own stopping rows

1. Split the fold's training rows: fit rows and stopping rows
2. Fit a fresh scaler on the fit rows only (fit_arrays)
3. Seed, then build the network
4. fit with early stopping on the stopping rows
5. **Score the held-out fold, never fitted or scaled on**

Out-of-fold outputs

One output per training row, from the network that did not see it. Pooled ROC AUC 0.7754, average precision 0.5343.

Threshold for recall 0.70: 0.1758 (out-of-fold recall 0.7002). No validation row used.

mlkit.assert_disjoint runs twice per fold: no held-out row in the fit or stopping rows.

How the threshold is chosen

1. Flag at or above

a cut on the output

2. Lower cut

more flags, more defaults caught

3. The rule

the highest cut that catches 70 percent

4. Out of fold

never the rows that report

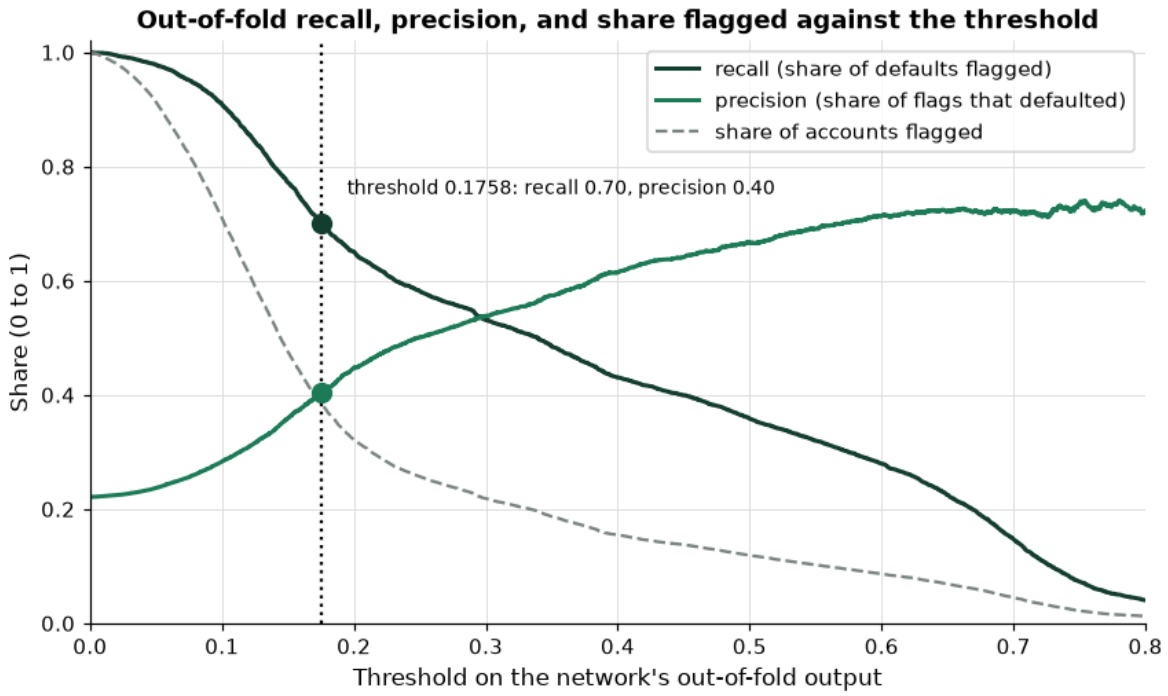
You control

The recall target.

What comes out

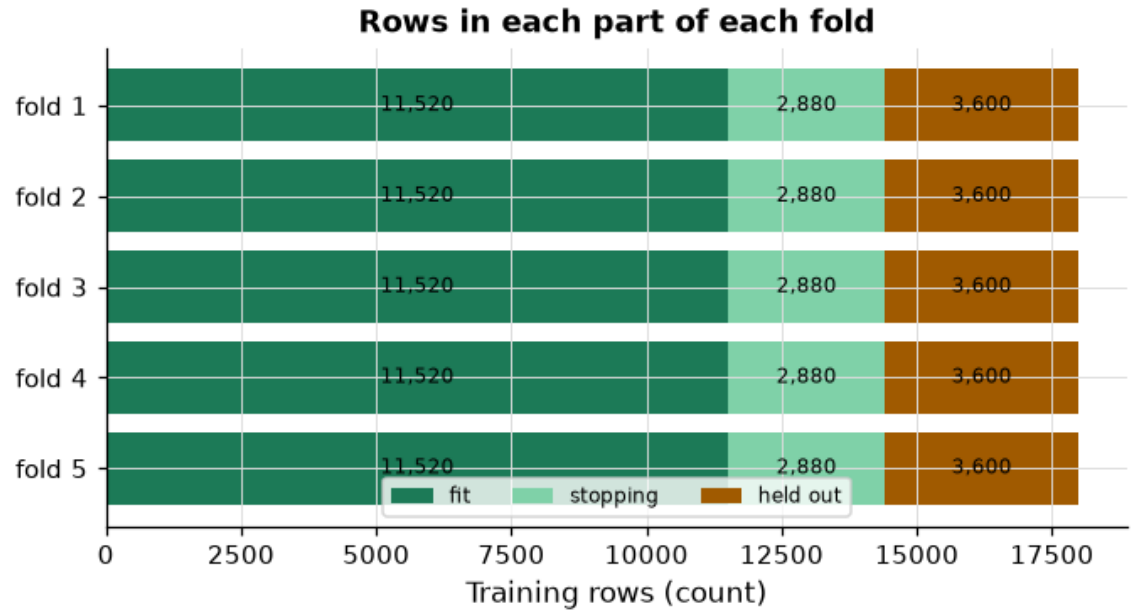
One threshold, with its precision and flagged count.

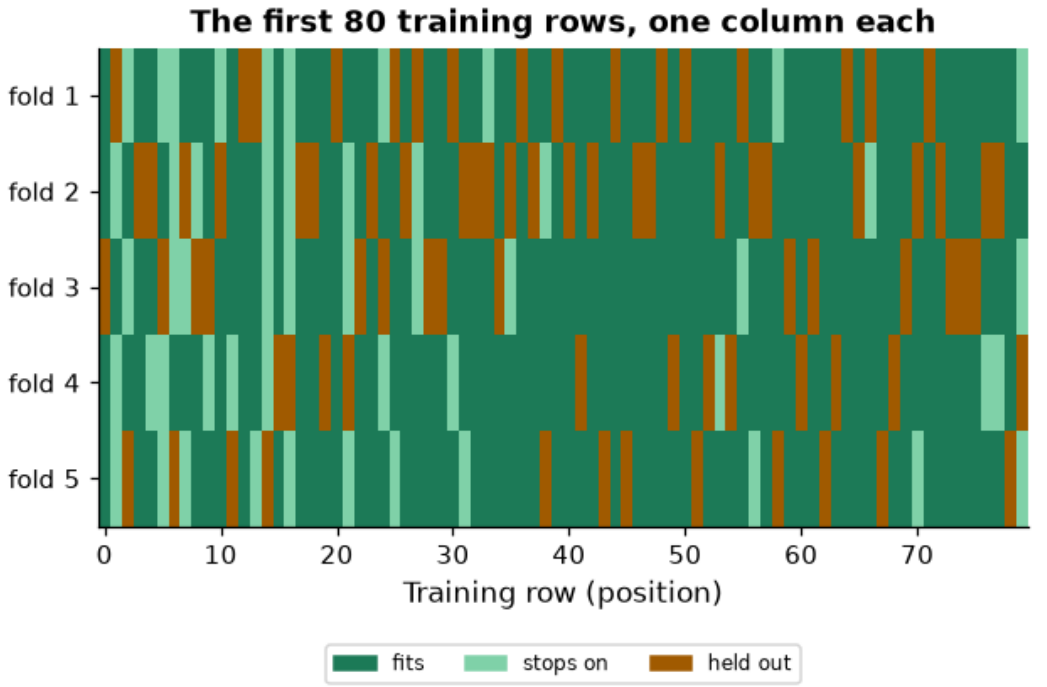
The threshold as a curve



The dot: threshold 0.1758, out-of-fold recall 0.7002. The curves trade recall for precision; which trade is right is the desk's call.

The five folds, drawn





Each fold holds out 3,600 rows; every training row is held out once, so the out-of-fold outputs have one entry per row.

Why the course scores with scikit-learn

0.775425

keras.metrics.AUC: 200 evenly spaced thresholds

0.775440

roc_auc_score: every distinct output

Close, not equal, and the gap changes with the outputs

Keras's defaults, read in its 3.15.1 source: num_thresholds=200, summation_method="interpolation". When models differ by thousandths, an approximation of unknown size is not good enough.

keras.metrics.AUC appears once, in a cell marked demonstration. Every score is scikit-learn's.

The case study: context, objective, rows

Objective

1. Choose the scaler by five folds
2. Measure the seed's spread first
3. One setting at a time, on stopping rows
4. Thresholds from out-of-fold outputs
5. Report on validation, with intervals
6. What it shows, and where it fails

Rows	Accounts	Defaults
fit	14,400	3,186
stopping	3,600	796
validation	6,000	1,327
test	closed	never scored

Every part 22.12 percent defaults: every cut is stratified.

Does a small network rank the accounts better than the two models already on the desk, and where does it fail?

How the two scalers differ

1. Standardize

mean and spread: a long tail stays long

2. Quantile

ranks mapped onto a normal curve

3. Same folds

one cv_network run per scaler

4. Choose

by pooled out-of-fold average precision

You control

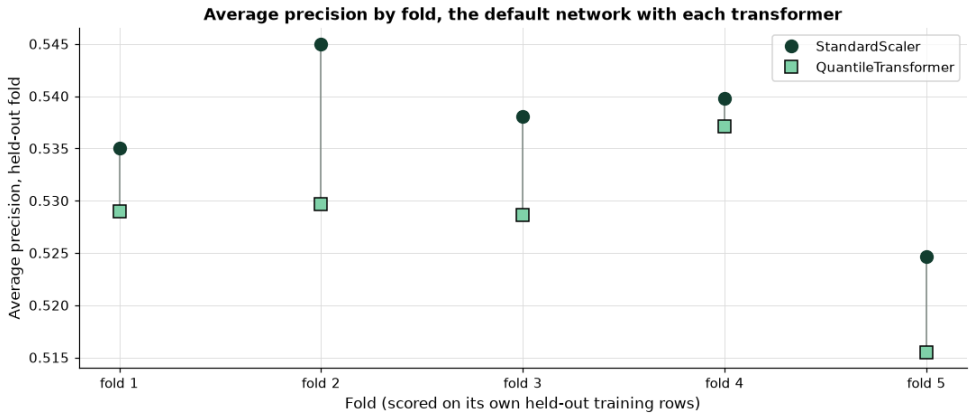
The scaler.

What comes out

A fold table for each, and a choice.

Preparation: two ways to scale

Transformer	Pooled AP	Pooled ROC AUC	Fold sd of AP
StandardScaler	0.5343	0.7754	0.0068
QuantileTransformer	0.5258	0.7710	0.0070

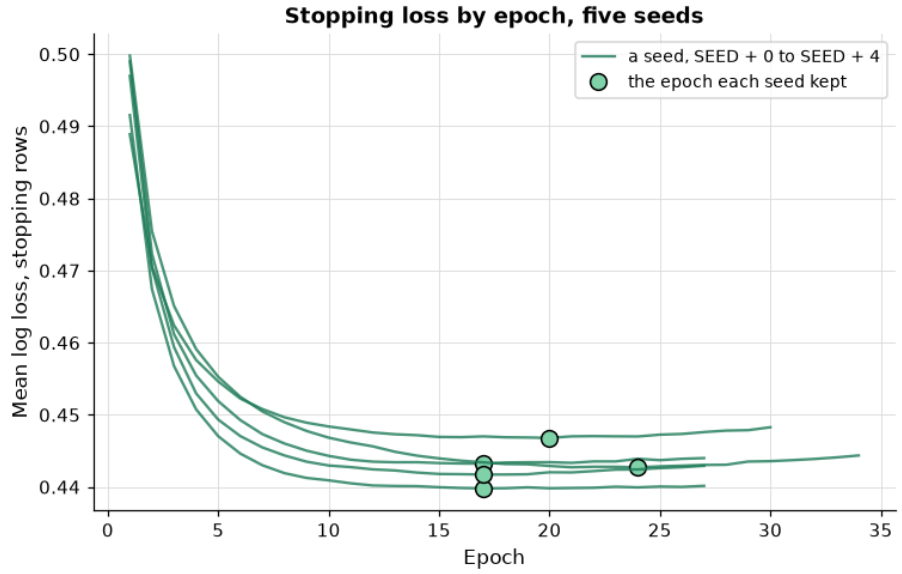


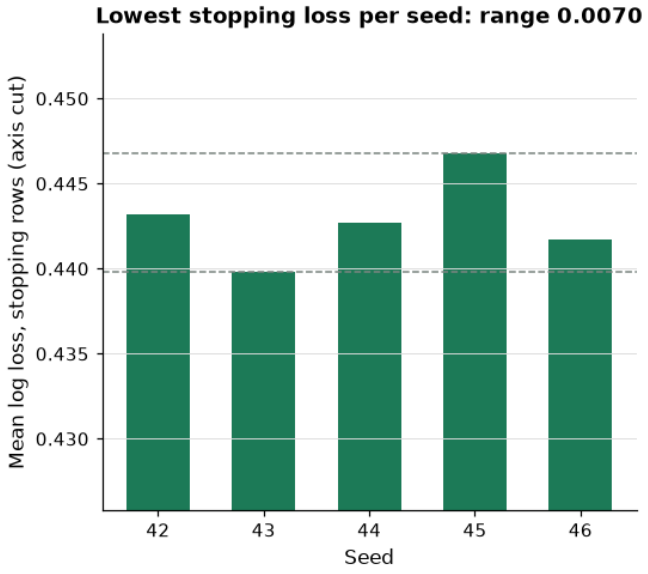
Why a quantile scaler

The card amounts are long-tailed. A quantile transformer maps each value's rank among the fit rows onto a normal curve, so nothing sits far out.

StandardScaler kept: higher on both measures, ahead on 5 of 5 folds. Chosen by five folds inside the training rows.

How much the seed alone moves a network





0.0070

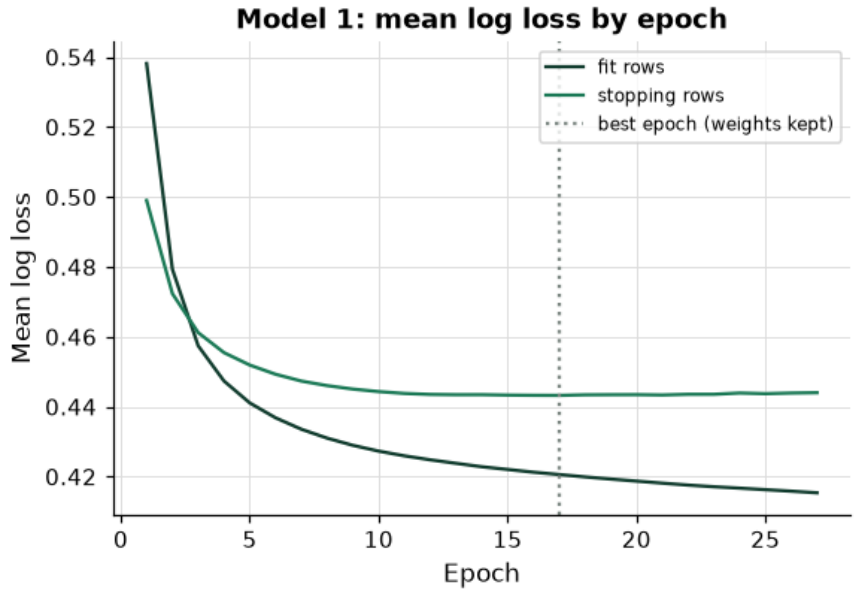
range of the lowest stopping loss across five seeds

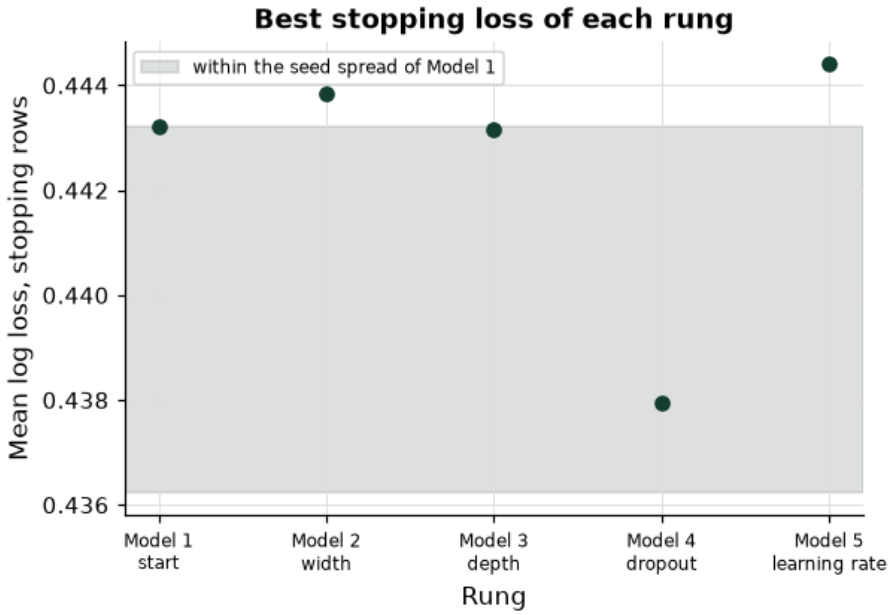
The bar for the ladder

A change is kept only if it lowers the best stopping loss by more than 0.0070.

Same network, same rows: best epochs 17 to 24; only the starting weights and the order of the batches changed.

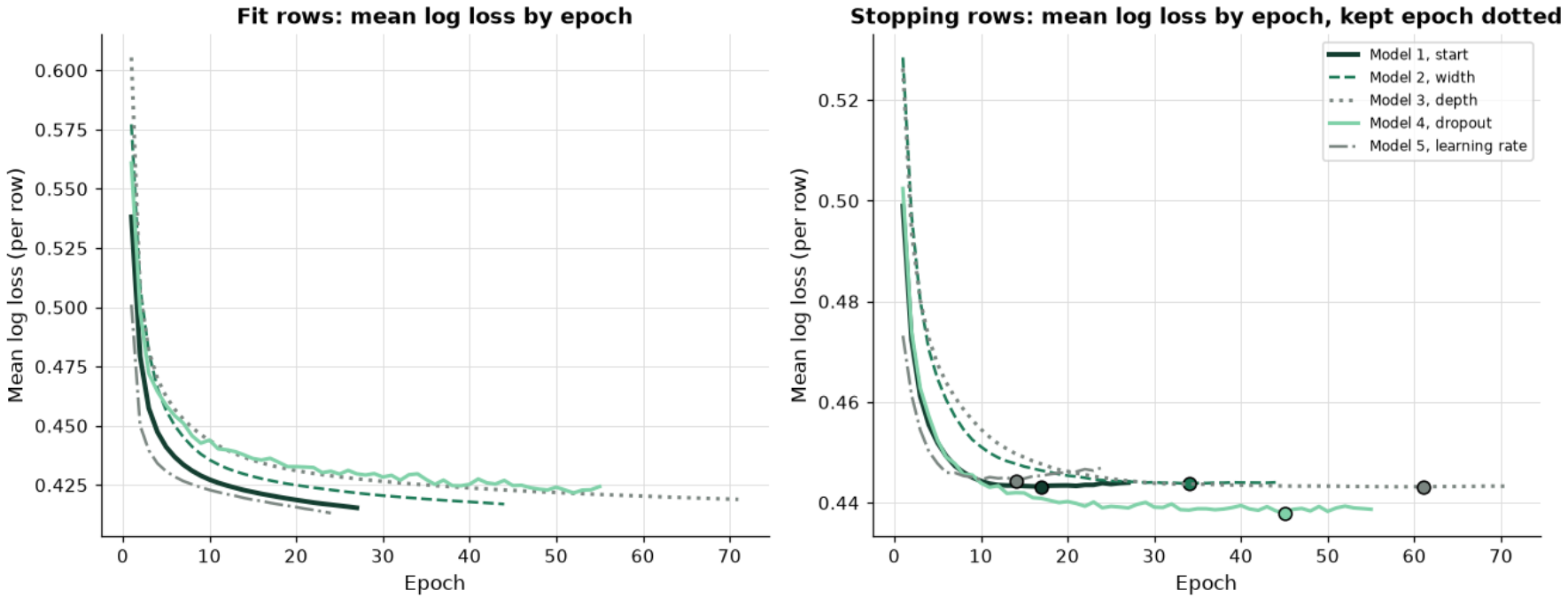
The ladder: one setting at a time





No rung beat the seed spread: dropout came closest (0.4379 against 0.4432). Model 1 stands.

Every rung's loss curves



The five lowest stopping losses lie within about 0.0065 of each other, less than the seed spread of 0.0070.

Thresholds from out-of-fold outputs

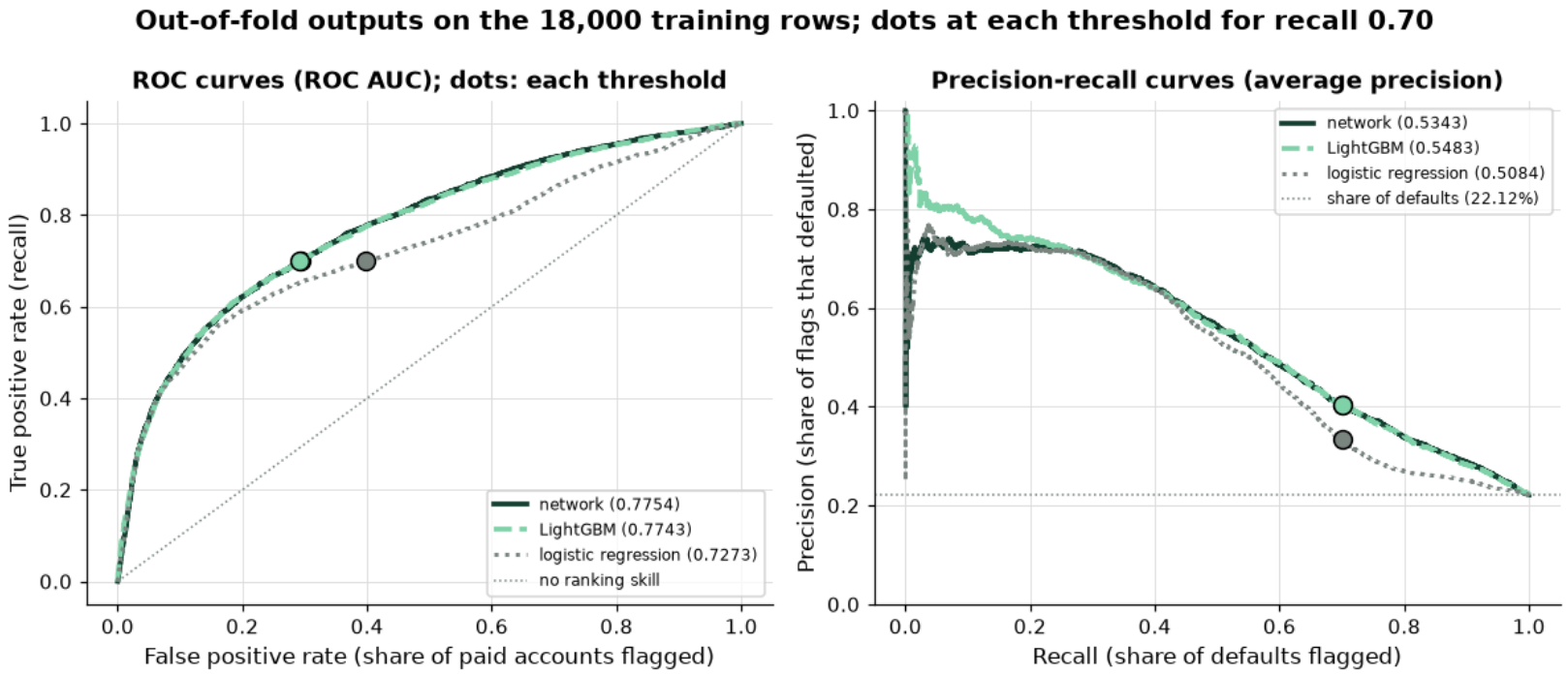
Model	Threshold	OOF ROC AUC	OOF avg precision
network	0.1758	0.7754	0.5343
LightGBM	0.1778	0.7743	0.5483
logistic regression	0.2047	0.7273	0.5084

The two measures disagree at the top

Out of fold, the network leads on ROC AUC (0.7754 against 0.7743) and LightGBM on average precision (0.5483 against 0.5343).

Each threshold: the highest whose out-of-fold recall reaches 0.70. No validation row chose any of them.

Out-of-fold curves of the three models



Every dot sits at a recall of 0.70; the right panel shows the precision each model has there.

How a bootstrap interval works

1. Resample

draw the validation rows with replacement

2. Score

the model on each resample

3. Repeat

1,000 times; keep the middle 95 percent

4. Differences

read the same way on the same resamples

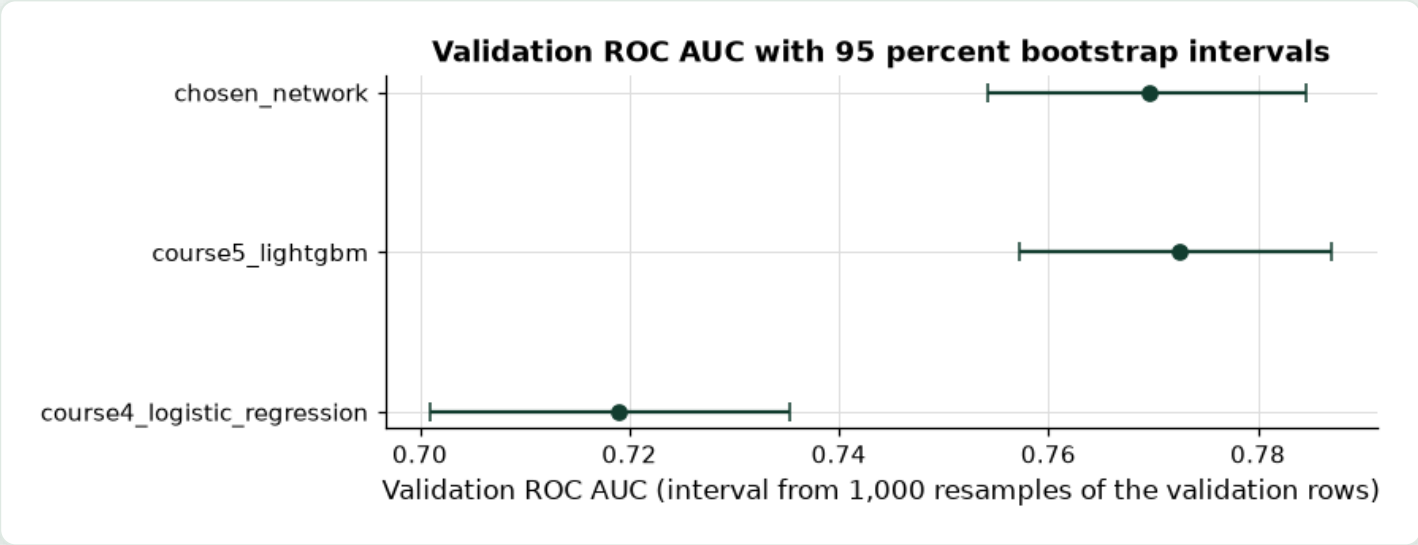
You control

The number of resamples, the level, and the seed.

What comes out

A lower and an upper bound for each score and each difference.

Validation rows, with intervals

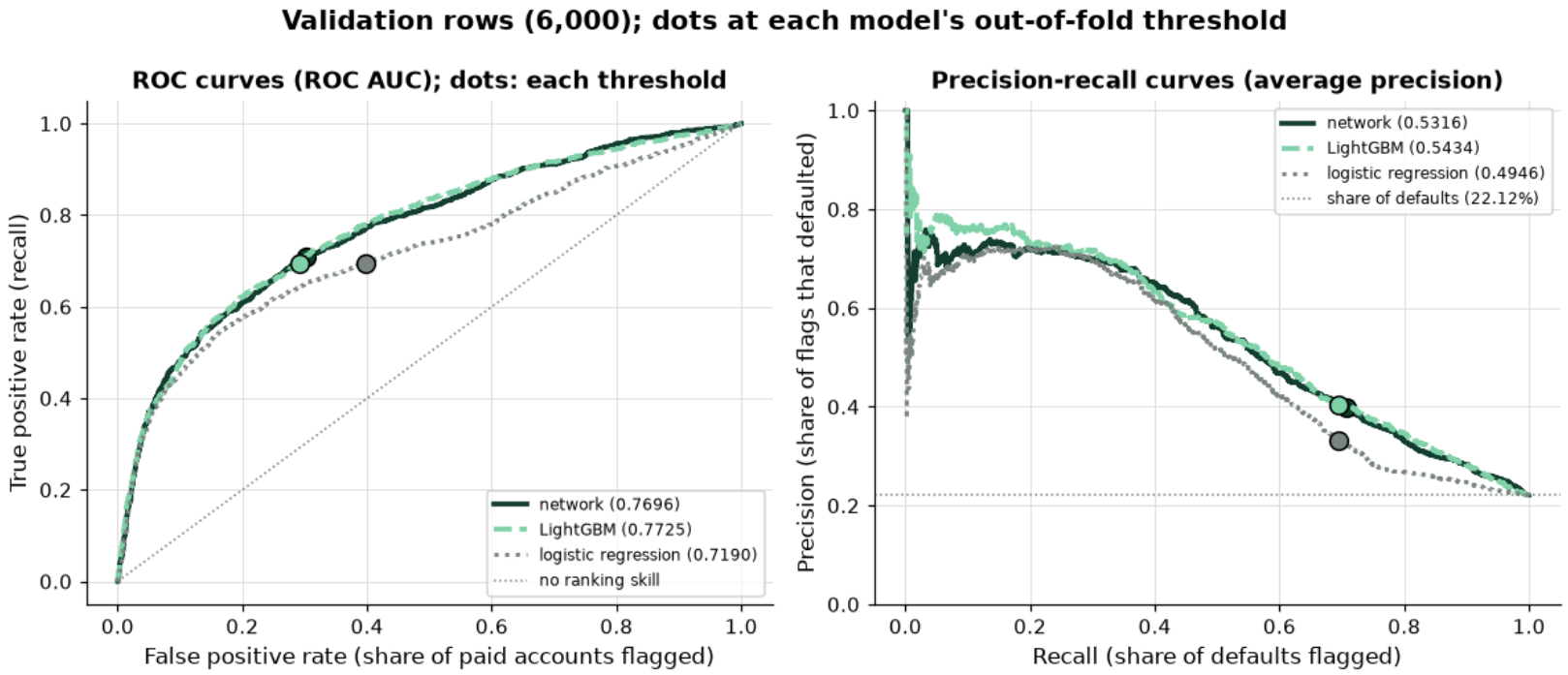


Model	ROC AUC	Recall
network	0.7696	0.7076
LightGBM	0.7725	0.6956
logistic regression	0.7190	0.6956

Recall at the out-of-fold thresholds: near 0.70, not on it. Nothing is moved.

LightGBM 0.7725 (0.7572 to 0.7871), the network 0.7696 (0.7542 to 0.7847): the order at the top flipped from out of fold.

Validation curves of the three models



Recall at the dots: network 0.7076, LightGBM 0.6956, logistic regression 0.6956. The thresholds were set on other rows.

The paired differences

Network minus	Difference	Lower	Upper	Network higher
LightGBM	-0.0029	-0.0109	0.0048	0.2370
logistic regression	0.0506	0.0378	0.0635	1.0000

Against LightGBM: holds zero

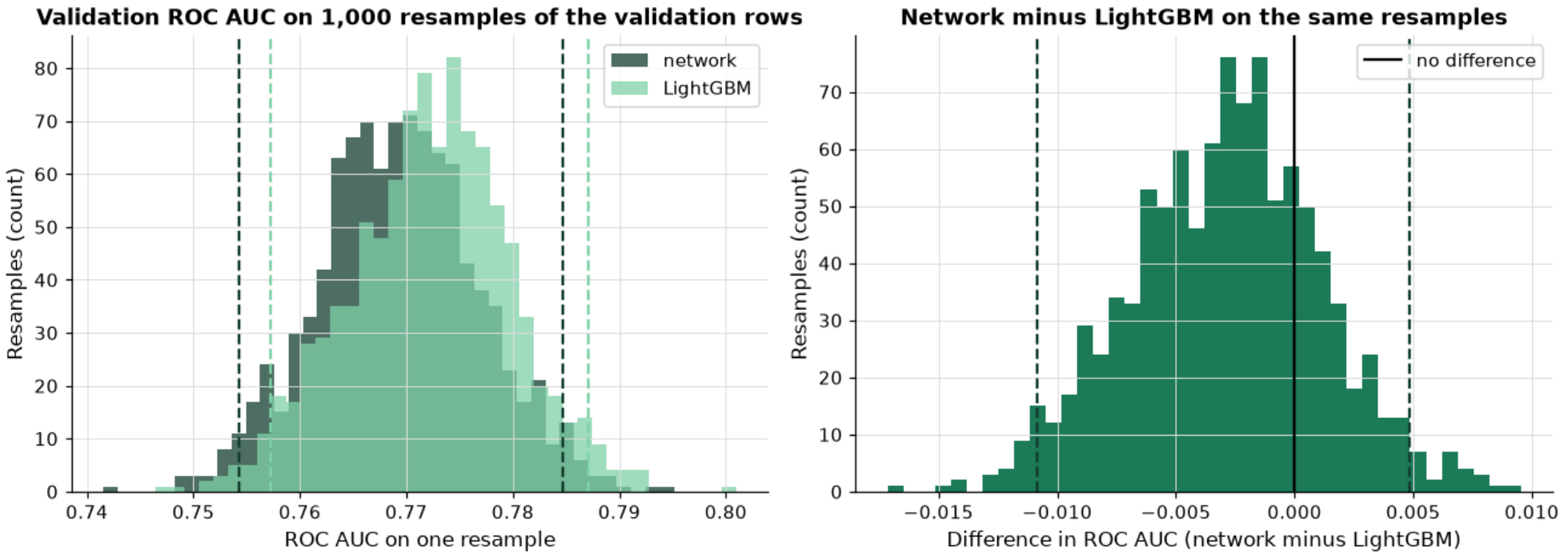
On these rows the two cannot be told apart by ROC AUC.

Against logistic regression: above zero

The network is higher on every resample.

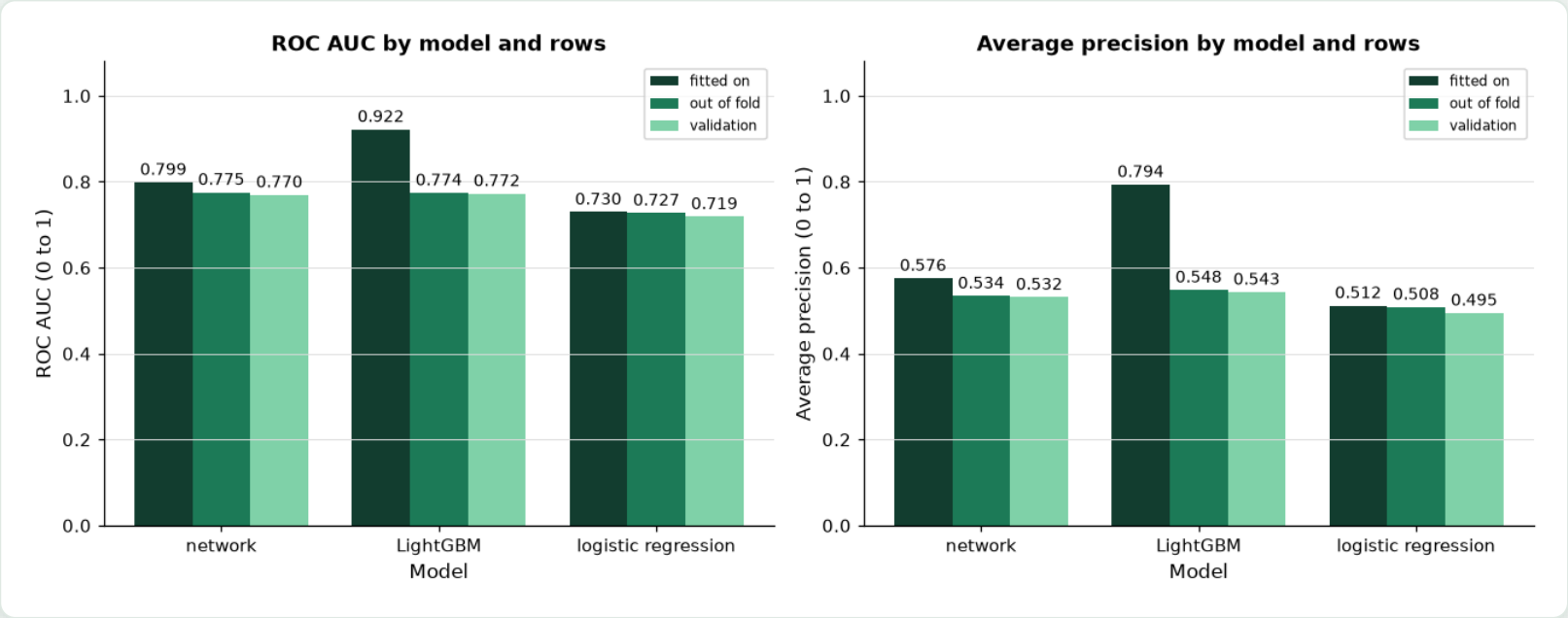
Same 1,000 resamples for all three. Intervals cover which rows were scored, not which seed was used.

The resamples, drawn



Network minus LightGBM: -0.0109 to 0.0048, holding zero. These rows cannot say which of the two ranks better.

Every model on every set of rows

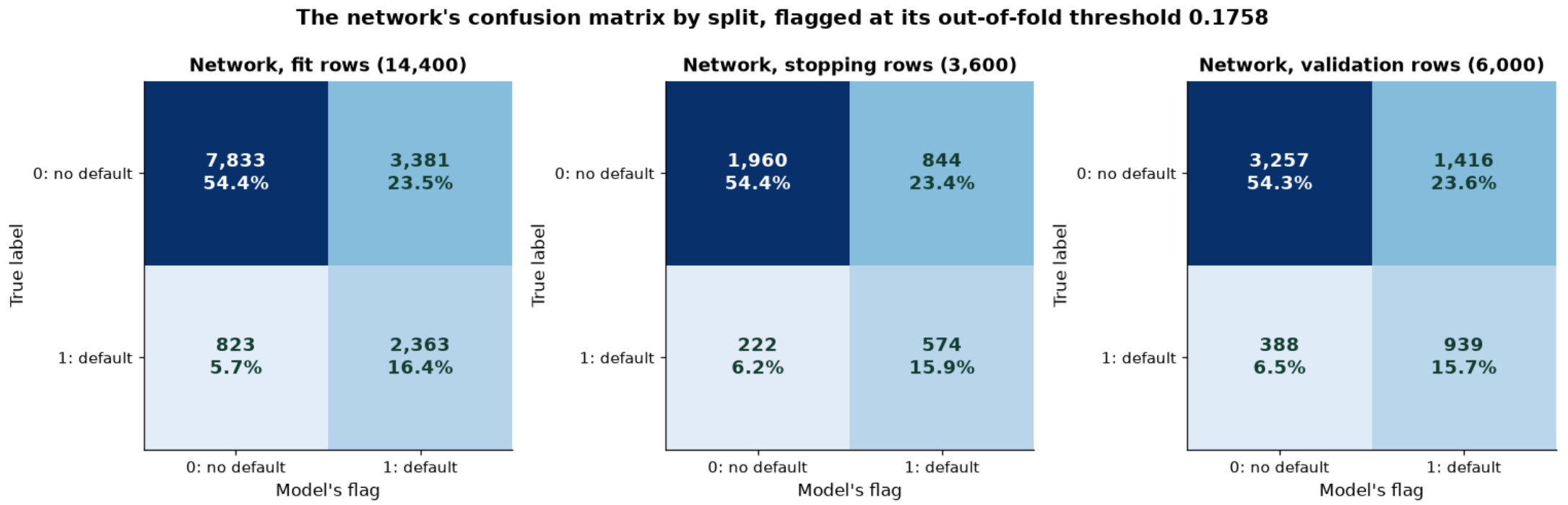


ROC AUC	Fitted	Valid.
network	0.7988	0.7696
LightGBM	0.9220	0.7725
logistic regression	0.7296	0.7190

A score on a model's own fitted rows measures fit; no choice today was made on one.

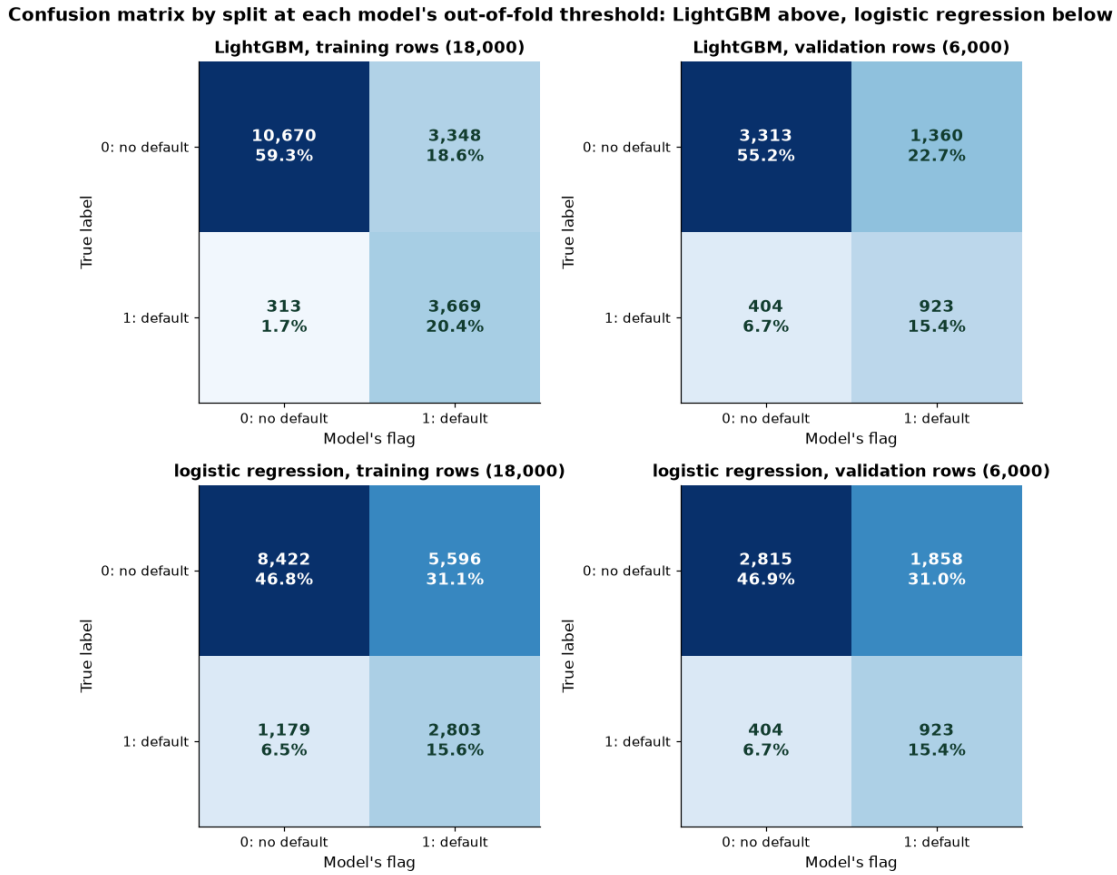
Out of fold and on validation, the network and LightGBM are close; the logistic regression is lower on both measures.

The network's confusion matrices



Recall: fit rows 74.2%, stopping rows 72.1%, validation rows 70.8%.

LightGBM and the logistic regression

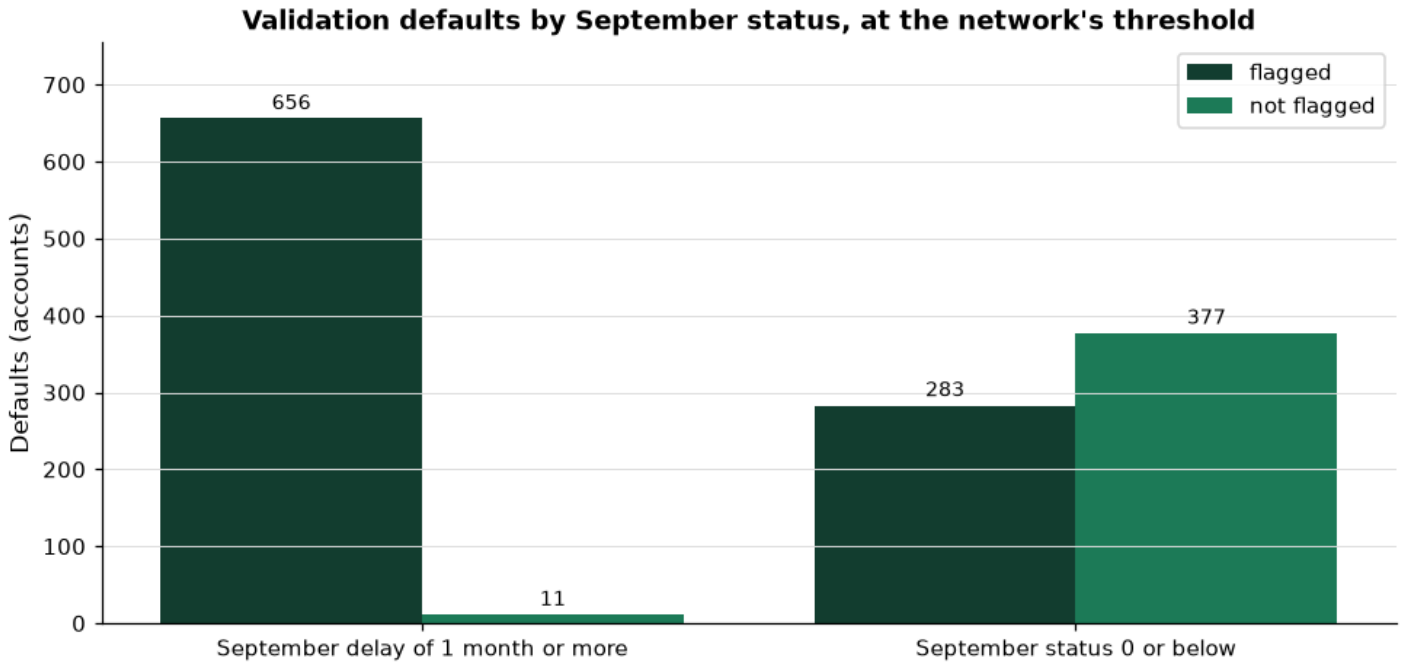


Recall: training to validation

LightGBM 92.1% to 69.6%; logistic regression 70.4% to 69.6%.

Three hundred trees fit their own rows closely; the validation panels are the ones to compare.

Where the network misses



656 of 667

defaults with a delay, flagged

283 of 660

defaults without one, flagged

Two groups of nearly the same size, two very different pictures.

What the model shows, and where it fails

What it shows

Validation ROC AUC 0.7696: above the logistic regression beyond the noise; level with LightGBM within it.

Where it fails

Flags 656 of 667 defaults with a recent delay, 283 of 660 without. 200 defaults missed by all three; outputs correlate 0.9198 with LightGBM's.

Allowed names

model probability of default in this file

flagged at threshold t

the model's score (class-weighted)

Never called

a credit decision

an approval or a decline

a risk grade or credit score

Excel side by side

Excel side by side (tested in Excel)

`=IF(B2>=$F$1,1,0)`	the flag, in C2, filled down
`=COUNTIFS(A2:A6001,1,C2:C6001,1)`	defaults flagged: 939
`=COUNTIFS(A2:A6001,0,C2:C6001,1)`	non-defaults flagged: 1,416
`=F3/(F3+F4)` and `=F3/(F3+F5)`	precision 0.3987, recall 0.7076
`=PERCENTILE.INC(A2:A1001,0.025)` and `0.975`	network 0.7542 to 0.7847
`=PERCENTILE.INC(D2:D1001,0.025)` and `0.975`	minus LightGBM -0.0109 to 0.0048
`=COUNTIF(D2:D1001,">0")/COUNT(D2:D1001)`	network higher: 0.2370

No network training, cross-validation, or seeded bootstrap in Excel. Every number above equals Python's within 1e-12.

Save the week: git tag week1

1. Code

netkit with today's two functions

2. cv rows

out-of-fold, on training rows

3. Validation

three rows; git tag week1

```
git diff week1~1 week1 -- results.csv
+chosen_network,validation,0.7696,0.5316,0.1758,0.6993,0.3987,0.7076,0.5100
+course5_lightgbm,validation,0.7725,0.5434,0.1778,0.7060,0.4043,0.6956,0.5114
+course4_logistic_regression,validation,0.7190,0.4946,0.2047,0.6230,0.3319,0.6956,0.4494
```

24 tests pass. week1~1 is the commit before week1; a tag works anywhere a commit does.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows stop, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the threshold is chosen on the validation rows: 0.1786, recall 0.7001 by construction.

Code that runs is not the same as code that is right.

What you can do now

- Build and seed a network with dense_classifier
- Cross-validate it with cv_network; threshold from out-of-fold
- Measure the seed's spread before comparing settings
- Compare with paired bootstrap intervals
- Write what it shows, and where it fails

Week 2, Day 6 Loss functions and the learning rate, on the benchmark bonds